

Introduction to Python Loops - Practical Assignment

Learning objectives

By the end of this assignment you should be able to explain the difference between a for loop and a while loop, choose the right loop for a given problem, and use the range() function together with accumulator variables to compute running totals. You should also be comfortable reading a short program and predicting its output before running it.

Background

A loop executes a block of statements repeatedly until a particular condition is satisfied. Python offers two kinds of loop. The for loop iterates over any iterable object such as a list, a string or the sequence produced by range(). The while loop keeps running as long as its condition evaluates to True, which makes it the natural choice when the number of iterations is not known in advance, for example when reading user input until a sentinel value appears.

Inside a loop body you will often see an accumulator: a variable initialised before the loop (usually to 0 or to an empty list) and updated on every iteration. The classic example is summing the numbers from 1 to n. A common mistake is to reset the accumulator inside the loop, which silently discards all previous work and produces a result that is only based on the final iteration.

Two keywords change the normal flow of a loop. The break statement leaves the loop immediately, skipping any remaining iterations. The continue statement skips the rest of the current iteration and jumps straight to the next one. Both are useful, but overusing them makes code harder to follow, so prefer a clear loop condition when one exists.

Tasks

Task 1. Write a function sum_to(n) that returns the sum of all integers from 1 to n inclusive using a for loop and range(). Do not use the built-in sum() function. Test it with n = 10 (expected result 55) and n = 100 (expected result 5050).

Task 2. Write a function count_vowels(text) that uses a for loop over the characters of the string and returns how many of them are vowels. Treat upper-case and lower-case letters the same way.

Task 3. Write a program that repeatedly asks the user for a number and stops when the user types the word "done". Use a while loop. When the loop finishes, print the total, the count and the average of the numbers entered. Make sure the program does not crash when no numbers were entered at all.

Task 4. Read the following program carefully and write down what it prints without running it. Then run it and compare. The program prints the value of `i` for every `i` in `range(1, 20)` that is not divisible by 3, and stops entirely once `i` reaches 13.

Common pitfalls to watch for

Off-by-one errors are the most frequent bug in loop code: remember that `range(1, n)` stops at `n - 1`, so summing from 1 to `n` inclusive needs `range(1, n + 1)`. Infinite while loops usually mean the variable in the condition is never updated inside the body. Finally, indentation determines which statements belong to the loop body in Python, so a single misplaced line can move a print statement in or out of the loop.

Submission

Submit a single Python file containing the three functions and the program from Task 3, plus a short comment block with your prediction and observation from Task 4. Hints are available from the assistant if you get stuck; start by describing what you expected to happen and what actually happened.